Messaging: leader-elected processors and a partitioned read model

What was built, how it works and what the load runs showed. Branch `feat/messaging`, session of 10 October 2026, steps 3 and 6 of the tuning order in `doc/messaging-architecture.md` section 6.

1. Summary

	Before	After
Several nodes	every node ran every projection and relay: the work was done N times and the live topic carried N copies of each record, so every socket received every message N times	each group and partition runs on exactly one node, elected through a lease in MySQL; another node takes over within 10 s of a crash
Read model under a burst	one thread, about 23 events/s; a burst of 4,800 messages left the read model 42 s behind the store	four threads for the conversation projection; the lag stayed below a third of a second in the same burst
Connection pool	20, enough but tight	40
Posts lost	none (since 1 October)	none, including during the crash of the leader

The two changes are framework work in `cqrs-reactive` (an R2DBC lock registry and partition seeding), switched on in the messaging service by configuration. Nothing is pushed or committed; the branch is ready for a commit when you ask.

2. The problem

The read side of a service is a set of OpenCQRS event handling processors, one per group: here `conversation-read-model` (writes the MySQL tables), `messaging-public` (relays public events to Kafka) and `messaging-live` (relays every conversation event to the live topic that pushes to WebSocket clients). Two things were wrong with how they ran.

- **Every node ran every processor.** OpenCQRS's default life-cycle controller starts each processor with the application context. Two nodes meant two projections of every event (idempotent upserts, so the data stayed right, but MySQL did the work twice) and two copies of every record on the live topic, which every node consumes to push to its sockets: each client saw every message twice. Run 3 below measures exactly that.
- **One thread projected every conversation.** The projection handler costs about 40 ms per event (a transaction with two or three statements, then the checkpoint), so one partition absorbs about 23 events a second. The state cache experiment of the previous report produced 4,800 messages in four minutes and the read model fell 54 s behind; run 1 below reproduces it at 42 s.

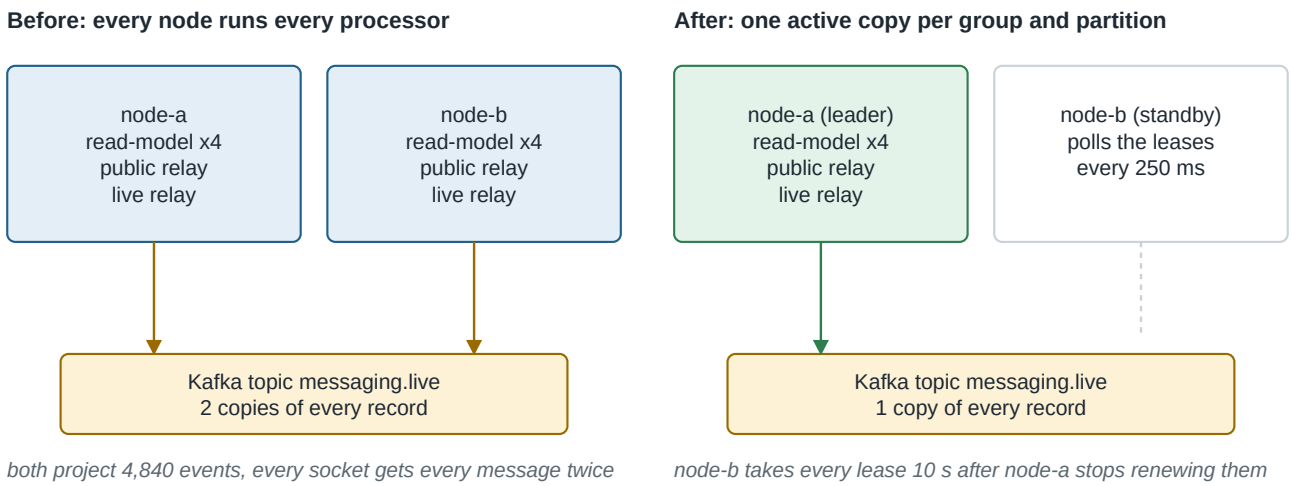


Figure 1. Two nodes with the old controller and with leader election.

3. How a processor is elected

OpenCQRS 1.0.0 already contains the alternative controller: with `opencqrs.event-handling.standard.life-cycle.controller=leader_election` it wraps every processor in a Spring Integration `LockRegistryLeaderInitiator`. That class runs one thread per processor which tries to obtain a lock named after the processor's role (group and partition) from a `LockRegistry`. The node that gets the lock is granted leadership and starts the processor; the others keep trying. While it is the leader it calls `tryLock` again every heartbeat (500 ms) to renew; when a renewal fails it is revoked and stops the processor. OpenCQRS only needs a `LockRegistry` bean, and the ones Spring Integration ships want JDBC, Redis or ZooKeeper, none of which this reactive stack has. So the framework got one over R2DBC and MySQL.

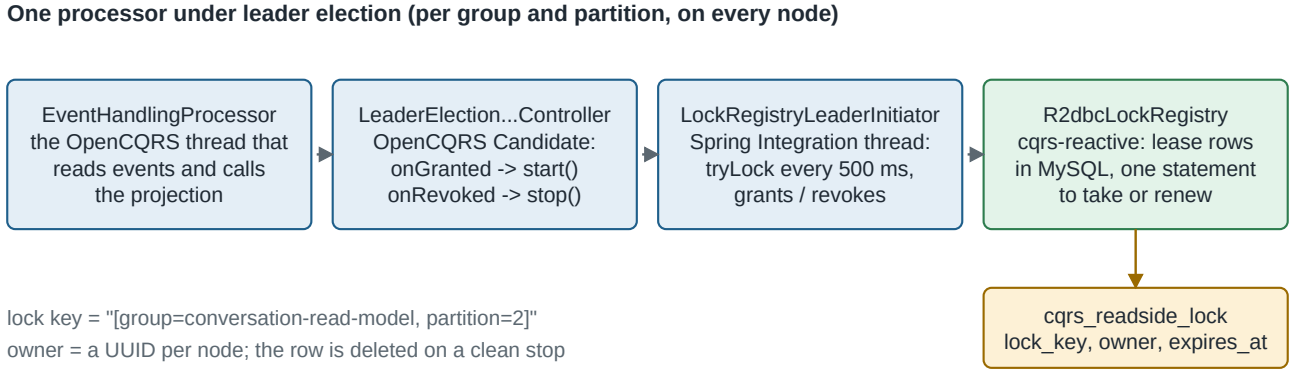


Figure 2. The chain from the processor to the lease row, on every node. The initiator calls tryLock again while it is the leader (that is the renewal) and follows the success with one unlock, so the lock keeps a hold count; a lease whose expires_at has passed can be taken by any node.

3.1 The R2DBC lock registry

`org.linlinjava.announce.cqrs.readside.R2dbcLockRegistry` keeps one row per lock key in the table `cqrs_readside_lock` of the service's own database, next to the checkpoint table:

```
CREATE TABLE cqrs_readside_lock (lock_key VARCHAR(190) PRIMARY KEY, owner VARCHAR(64) NOT NULL, expires_at TIMESTAMP(3) NOT NULL)
```

A lock attempt is one statement that takes the row when it is free, expired or already this node's, renews the lease in the same statement, and leaves it alone otherwise; a second statement reads back who owns it now:

```
INSERT INTO cqrs_readside_lock (lock_key, owner, expires_at) VALUES (:key, :me, NOW(3) +
ttl)
  ON DUPLICATE KEY UPDATE owner = IF(owner = :me OR expires_at < NOW(3), :me, owner),
    expires_at = IF(owner = :me, NOW(3) + ttl, expires_at);
SELECT owner FROM cqrs_readside_lock WHERE lock_key = :key;
```

MySQL evaluates the assignments left to right, so the lease is refreshed exactly when the owner column ends up being this node. Because the primary key serialises the two statements of competing nodes, exactly one of them reads back its own id (the test with six contenders checks that). Properties, all under `cqrs.readside.leader-election`:

Property	Default	Meaning
enabled	false	creates the registry bean cqrsLockRegistry; off, every node runs every processor as before (announce-management and order are untouched)
ttl	10s	how long a lease lasts without renewal: the longest a crashed node keeps its groups, so the failover gap
poll-interval	250ms	how often a node that does not hold a lease asks for it again
table / create-table	cqrs_readside_lock / true	the lock table, created on start-up when missing

- **Reentrant with a hold count.** The initiator calls `tryLock` while it already holds the lock (that is the renewal) and follows the success with one `unlock`. A naive lock would release the lease every heartbeat, so the lock counts holds and deletes its row only when the last hold is released. A failed renewal resets the count: the node learns it lost the lease.
- **Owner per node, not per thread.** The owner is a UUID created with the registry, one per node, so a lease survives a thread change on the same node and a clean shutdown releases every lease of the node (the end-to-end test observed the table empty after shutdown).
- **Blocking only where the framework already blocks.** The statements run through `DatabaseClient` and are awaited on the initiator's own thread, as the progress tracker does on the processor thread; the event loop is never involved.
- **Cost.** Two short statements per processor per node every heartbeat: about 4 statements a second per processor per node. Measured idle with one node and three processors: 12 statements a second on MySQL. Two nodes and six processors cost about 50 a second.

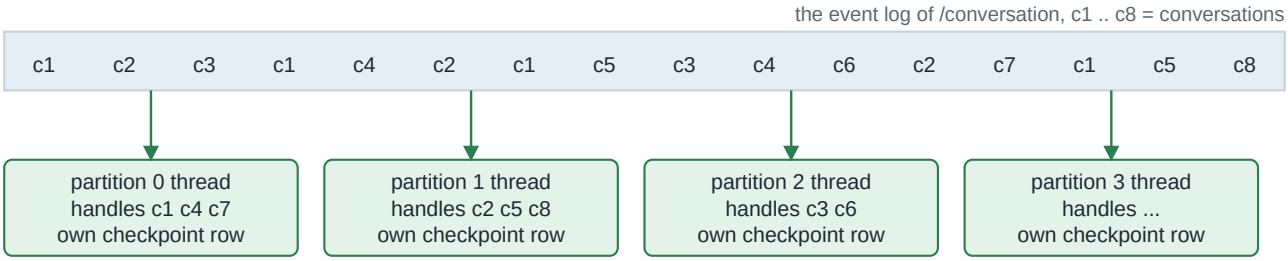
3.2 What leader election gives and what it does not

- **One active copy.** A group and partition runs on exactly one node; the others poll. Projection work and relay records are no longer multiplied by the node count.
- **Failover within the lease.** A node that dies stops renewing; after the ttl another node takes the lease and starts the processor from the shared checkpoint. Events in flight at the crash are handled again (at-least-once, as projections must tolerate anyway).
- **No load sharing.** Leases are first come first served: the first node up takes all of them and a second node is a warm standby until a lease moves. For this service that is fine, the projection needs far less than one node, but it is not a way to spread groups over nodes.
- **A tunable gap.** The ttl bounds how long the read model and the live push stand still after a crash. 10 s against a 500 ms heartbeat is conservative; the command side keeps acknowledging posts throughout, and the clients receive the pushed messages when the relay resumes.

4. Partitions for the read model

`opencqrs.event-handling.groups.conversation-read-model.life-cycle.partitions=4` starts four processor threads for the projection. OpenCQRS resolves the sequence of an event from the second level of its subject (`/conversation/{id}`), so every event of one conversation lands on the same partition and the order per conversation holds; different conversations are projected in parallel. Each partition has its own checkpoint row in `cqrs_readside_progress` and advances it past the events it skips as well, so an idle group shows four rows at the head. The two relay groups stay at one partition: one Kafka send per event is not the bottleneck.

conversation-read-model with 4 partitions: events spread by conversation, order kept



A partition without a checkpoint row would start at the first event; the framework seeds the missing rows from the group's lowest checkpoint on start-up.

Figure 3. Events spread over partitions by conversation.

4.1 Why a new partition is dangerous, and the seeding

OpenCQRS documents that the partition count must only be changed administratively, with every node stopped, because a partition that has no checkpoint starts at the first event of the log. For the projection that means a full replay (idempotent, but minutes of work); for a relay group it would republish every event to Kafka. The framework now closes that trap: on start-up, for every group configured with more partitions than it has checkpoint rows, `R2dbcProgressTracker.seedPartitions` inserts the missing rows with the group's lowest existing checkpoint. The lowest is never ahead of any partition, so no event is skipped; at worst a short replay when the group was not caught up when it stopped. A group with no row at all is left alone and starts from the beginning, which is what a brand-new projection should do. Property `cqrs.readside.progress.seed-partitions`, on by default. Run 2 started with one row at the store's head and logged *Seeded 3 new partition(s) of group 'conversation-read-model' from its lowest checkpoint (4 partitions configured)*. The count must still only change with every node stopped, as OpenCQRS requires.

5. Configuration of the messaging service

Setting	Value	Profile
cqrs.readside.leader-election.enabled / ttl / poll-interval	true / 10s / 250ms	main; test uses the table cqrs_readside_lock_test so a node on the same database cannot block the suite
opencqrs.event-handling.standard.life-cycle.controller	leader_election (was application_context)	main and test
opencqrs.event-handling.groups.conversation-read-model.life-cycle.partitions	4 (test: 2)	main and test
cqrs.readside.progress.seed-partitions	true	main (framework default)
spring.r2dbc.mybatis.pool.max-size	40 (was 20)	main

6. Tests

Test	What it proves
R2dbcStoresTest (cqrs-reactive-test, MySQL Testcontainer), lockRegistryGivesOneLeaseToOneOwnerUntilItExpiresOrIsReleased	a second node cannot take a held lease, even when it waits; the holder renews by locking again and the renewal-unlock keeps the lease; an unrenewed lease is taken over once expired and the old holder learns it; release frees the row at once; keys are independent
R2dbcStoresTest, lockRegistryContendersAgreeOnExactlyOneHolder	six registries race for one key on parallel threads: exactly one wins
R2dbcStoresTest, partitionSeedingCopiesTheLowestCheckpointToNewPartitionsOnly	ids compare as numbers (900 below 1200); only missing partitions are inserted; a group without rows is left alone; idempotent; one partition is never seeded
MessagingEndToEndTest, theProcessorsAreLeaderElectedAndTheReadModelIsPartitioned	the four processors of the test profile (read model in two partitions plus the two relays) run under the leader-election controller, each initiator is the leader of its role, the four leases in the lock table belong to this node, and the read model has a checkpoint for both partitions; the earlier cases of the class read the messages back in order from the partitioned projection

`mvn -pl cqrs-reactive,cqrs-reactive-test test -Dtest=R2dbcStoresTest`: 6 tests pass on the container. `mvn -pl messaging test`: 35 tests pass (Testcontainers for EventSourcingDB and Kafka, the read model in the live database).

7. Measurements

The load of the previous report: `ChatLoadDriver` opens 20 support conversations, prefills 200 messages into each over REST (4,000 posts, the burst), then 40 sockets post every 3 s for 60 s (800 posts). Every run writes 4,800 messages, 4,840 events for the read model. The shared dev EventSourcingDB on port 3000 had stopped answering streaming reads that afternoon (ping answered, every read-events call hung without a byte; its container needs a restart, which this session was not allowed to do), so the nodes ran against a fresh `thenativeweb/eventsourcingdb:1.2.0` container on port 3001 with a temporary data directory, the dev Kafka and the live MySQL. Load average 2 to 5. Before every run the three groups' checkpoints were set to the store's head.

7.1 Step 6: partitions and pool, one node (runs 1 and 2)

	run 1: 1 partition, pool 20 (old)	run 2: 4 partitions, pool 40 (new)
prefill of 4,000 messages over REST	169 s	192 s
posts sent / acknowledged / error frames	800 / 800 / 0	800 / 800 / 0
ack latency p50 / p95 / max	61 / 207 / 504 ms	48 / 127 / 179 ms
command latency PostMessage p50 / p95	64 / 234 ms	47 / 129 ms
projection handler, per event, p50	39 ms	41 ms
projection delay (read model behind the store) p50 / p95 / max	31.7 s / 42.4 s / 41.8 s	0.08 / 0.11 / 0.31 s
read model caught up after the driver ended	10 s later	already at the head
pool: connections pending	0	0

The single thread spent 209 s of handler time on 4,840 events: 23 events a second against an arrival rate of about 24 during the prefill, so the lag grew for the whole burst. Four partitions do not make an event cheaper (41 ms) but project four conversations at once; the backlog never exceeded a third of a second.

The posts got faster too (p95 from 207 to 127 ms): the commands' reads no longer waited on a pool kept busy by the projection. Pending connections were zero in both runs, so 20 was tight, not exhausted.

7.2 Step 3: two nodes, with and without leader election (runs 3 and 4)

Both nodes on this host (node-a on 8086, node-b on 8087), four partitions, pool 40, sockets on one node. Run 3 started both nodes with the old controller. Run 4 used leader election: node-a started first and took the six leases, node-b stayed idle, the driver's sockets were on node-b, and node-a was killed with SIGKILL 20 s into the socket phase, so its leases expired instead of being released.

	run 3: every node runs every processor	run 4: leader election, leader killed
posts sent / acknowledged / error frames	800 / 800 / 0	799 / 799 / 0
fan-out frames the sockets received (expected)	3,200 (1,600): every message twice	1,598 (1,598): every message once
projection handler calls, node-a + node-b	4,840 + 4,840 = 9,680 for 4,840 events	4,280 (until the kill) + 557 = 4,837 for 4,839 events
records seen on messaging.live per node	9,600 = 2 x 4,800	4,799
ack latency p50 / p95	89 / 222 ms	101 / 246 ms
fan-out latency p50 / p95 / max	93 / 229 / 374 ms	137 ms / 8.0 s / 9.7 s
projection delay p50 / p95 / max (node with sockets)	0.13 / 0.18 / 0.48 s	0.10 / 9.1 / 9.5 s
leases after the run	not used	all six on node-b

Run 4: the leader is killed with SIGKILL at t = 0 (clock of 2026-10-10 16:27)

Lease ttl 10 s: the gap is the ttl plus one poll. Fan-out max 9.7 s and projection delay max 9.5 s on node-b are this gap; nothing was lost.

Figure 4. The failover in run 4, from the logs of node-b.

Run 3 is the duplication the old controller caused, measured: each node projected all 4,840 events and relayed all 4,800 messages, the live topic carried two copies of every record, and every socket received every message twice. Run 4: node-b did nothing while node-a held the leases, the topic carried one copy, every socket received every message once. After the kill node-b was granted all six leases 9.8 to 10.1 s later and resumed from the shared checkpoints; the two events in flight at the kill were handled again by node-b, as at-least-once allows. During the gap the command side kept acknowledging (799 of 799), while the read model and the live push stood still: that gap is the 9.1 s p95 of the projection delay and the 8 s p95 of the fan-out latency, all delivered once the relay resumed. Nothing was lost.

8. Operating notes

- **Changing the partition count.** Stop every node of the service, change the property, start one node and read the *Seeded N new partition(s)* line, then the others. Lowering the count leaves stale rows for the removed partitions; delete them.
- **The failover gap is the ttl.** Lower `cqrs.readside.leader-election.ttl` for a shorter gap; keep it several heartbeats (500 ms) above the slowest MySQL round trip you expect, or a healthy leader will lose its leases.
- **Checkpoints after a test run** hold ids of the test container, one row per partition. Before starting a dev node, set partition 0 of each group to the dev store's head and delete the other partition rows; the seeding fills them.
- **Running two nodes locally.** Start the second with `--server.port=8087` and `ANNOUNCE_NODE=node-b`; `select lock_key, owner from cqrs_readside_lock` shows who runs what. A clean stop releases the rows at once.
- **The suite and a running node** share the live database. The test profile uses its own lock table so a node's leases cannot block the suite, but the checkpoint rows and the read-model tables are still shared (the known stall).
- **The dev EventSourcingDB** on port 3000 needs `docker restart announce_esdb`: its streaming reads hang. Its data is in a mounted volume.

9. Files changed

File	Change
cqrs-reactive/.../readside/R2dbcLockRegistry.java	new: the Spring Integration LockRegistry over MySQL through R2DBC (lease, reentrant hold count, holders() for diagnostics)
cqrs-reactive/.../readside/ReadSidePartitions.java	new: reads the configured partitions per group from the environment
cqrs-reactive/.../readside/R2dbcProgressTracker.java	checkpoints(group) and seedPartitions(group, partitions)
cqrs-reactive/.../readside/ReadSideAutoConfiguration.java	lock registry bean behind cqrs.readside.leader-election.enabled; partition seeding when the tracker is built
cqrs-reactive/.../CqrsProperties.java	LeaderElection settings; progress.seed-partitions
cqrs-reactive/src/main/resources/cqrs-schema-mysql.sql	DDL of cqrs_readside_lock
cqrs-reactive-test/.../R2dbcStoresTest.java	three new tests (lock registry, contention, seeding)
messaging/src/main/resources/config/application.yml	leader election on, controller leader_election, 4 partitions for the read model, pool 40
messaging/src/test/resources/application-test.yml	leader election with the test lock table, 2 partitions
messaging/.../MessagingEndToEndTest.java	cleanup of the test lock table; the leader election and partition case
doc/cqrs-reactive.md, doc/messaging-architecture.md, doc/messaging-context.md	framework section 2 (partitions, seeding, leader election); architecture 2.5, 3, 4.2, 4.6, 6 and the new 7c; context section 7

Branch state: all of the above is uncommitted on `feat/messaging`, on top of the three commits of the morning (state cache, merge, report). Next candidates in the tuning order: step 4 (Kafka latency settings on the live topic), step 5 (bounded sink and socket timeouts), step 7 (inbox query batching).